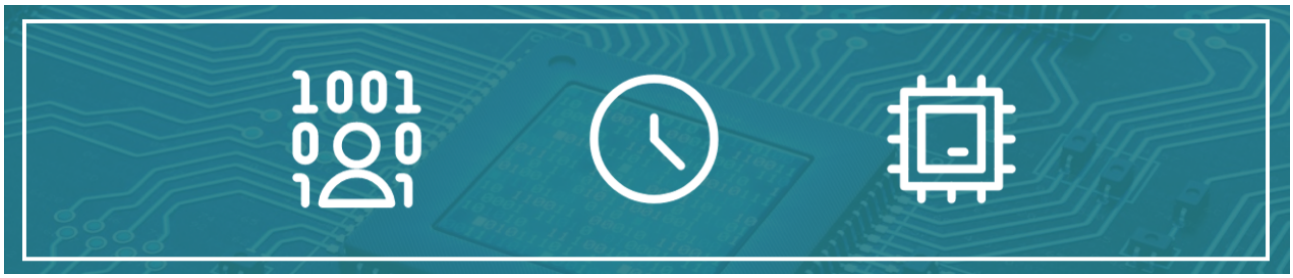
ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

ING4
Groupe 5

RAPPORT DE TP

TP Big Data

Trino : labs 2 et 3



Ruben COMBE--TAMAIN

J'atteste que ce travail est original,
qu'il est le fruit d'un travail individuel
et qu'il a été rédigé de manière autonome.

Paris, le 09/11/2026

Table des matières

1 Environnement et données **3**
1.1 Infrastructure utilisée . . . 3
1.2 Données . . . 3

2 Lab 2 : de silver à gold avec Trino **4**
2.1 Connexion, catalogues et types . . . 4
2.2 Requêtes sur la couche silver . . . 5
2.3 Métadonnées Iceberg . . . 6
2.4 Plan et exécution d'une requête . . . 6
2.5 Couche gold . . . 8
2.6 Validation . . . 9
2.7 Rafraîchissement et time travel . . . 9
2.8 Comparaison avec Spark . . . 10

3 Lab 3 : schéma en étoile **12**
3.1 Dimensions . . . 12
3.2 Table de faits . . . 12
3.3 Requêtes sur l'étoile . . . 12
3.4 Validation . . . 13
3.5 Exercices . . . 14
3.5.1 Exercice 1 : partitionnement . . . 15
3.5.2 Exercice 2 : cohortes par décennie de naissance . . . 15
3.5.3 Exercice 3 : INSERT, MERGE et time travel . . . 15
3.5.4 Exercice 4 : `optimize` . . . 16
3.5.5 Exercice 5 . . . 16

4 Bilan **16**

5 Annexes **17**
5.1 A. Configuration de la réplique . . . 17
5.2 B. Initialisation du catalogue et de la couche silver . . . 20

I. Environnement et données

A. Infrastructure utilisée

Les deux labs ont été exécutés sur une **réplique locale** de la plateforme, en conteneurs Docker, et non sur le cluster Kubernetes Adaltas (Onyxia). Trois services tournent (Figure 1) :

- **Trino 480**, coordinateur seul, même version que le chart Helm du lab de déploiement ;
- **Apache Polaris 1.8.0**, catalogue Iceberg REST avec authentification OAuth2 ;
- **SeaweedFS**, stockage compatible S3 (bucket `lab`), à la place du S3 de la plateforme.

```
terminal — environnement Docker
$ cd /tmp/claude-0/lab && docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
SERVICE   IMAGE                        STATUS                  PORTS
polaris   apache/polaris:latest        Up 3 minutes            8080/tcp, 8443/tcp, 0.0.0.0:8181-8182->8181-8182/tcp
s3        chrislusf/seaweedfs:latest   Up 3 minutes            7333/tcp, 8080/tcp, 8888/tcp, 9333/tcp, 18080/tcp, 18888/tcp, 19333/tcp, 0.0.0.0:8333->8333/tcp
trino     trinodb/trino:480            Up 3 minutes (healthy)  0.0.0.0:8080->8080/tcp
```

Figure 1 – Conteneurs de la réplique (`docker compose ps`).

Le catalogue Polaris `lab` pointe vers `s3://lab/warehouse` (Figure 2). Trino y accède par le catalogue `lakehouse` (connecteur Iceberg, type REST, OAuth2), comme dans le lab de déploiement. Les fichiers de configuration sont en annexe A.

```
terminal — catalogue Polaris et Trino
$ TOKEN=$(curl -s localhost:8181/api/catalog/v1/oauth/tokens -d grant_type=client_credentials -d client_id=root -d client_secret=s3cr3t -d scope=PRINCIPAL_ROLE:ALL | jq -r .a…
{"name":"lab","type":"INTERNAL","storage":"S3","endpoint":"http://s3:8333","allowedLocations":["s3://lab/warehouse"]}

{"nodeId":"659b02fb9e67","state":"ACTIVE","nodeVersion":{"version":"480"},"environment":"docker","coordinator":true,"coordinatorId":"y4x8r","starting":false,"uptime":"2.99m"}
```

Figure 2 – Catalogue Polaris `lab` et état du serveur Trino.

B. Données

Les CSV viennent du générateur du lab `uv` (Faker, graine 42), recopié tel quel. Leur taille correspond exactement à celle donnée dans les labs (7 351 et 331 568 octets, Figure 3). Le jeu contient **50 utilisateurs et 2 829 commandes**.

Écart avec l'énoncé. Les labs annoncent 5 000 commandes. Ce n'est pas ce que produit le générateur : `orders.csv` contient 2 829 lignes, pour 331 568 octets (environ 117 octets par ligne). Tous les comptages de ce rapport sont donc 2 829 et non 5 000.

```
terminal — jeu de données du lab 03
$ cd /tmp/claude-0/lab/gen && wc -c users.csv orders.csv && head -c 330 users.csv && echo && head -3 orders.csv && echo && python3 -c "import csv;print('lignes orders.csv :',…
  7351 users.csv
331568 orders.csv
338919 total
uuid,username,name,sex,address,mail,birthdate
bdd640fb-0667-4ad1-9c80-317fa3b1799d,garzaanthony,Charles Garcia,M,"908 Jennifer Squares
Robinsonshire, KY 01352",helenpeterson@gmail.com,1935-10-31
17be3111-1a2a-43ed-962b-0f79c37459ee,blairamanda,Ryan Munoz,M,"Unit 6184 Box 9593
DPO AP 09617",stanleykendra@gmail.com,2010-01-26
4
uuid,user_uuid,date,quantity,product
2339ba19-2563-4cc3-8a97-ebf555d596af,bdd640fb-0667-4ad1-9c80-317fa3b1799d,2020-01-01 00:02:17.891488+00:00,4,drink
b49e04cc-c243-49e7-a387-eaf6d4c3a832,bdd640fb-0667-4ad1-9c80-317fa3b1799d,2020-01-01 01:28:08.796458+00:00,5,bread

lignes orders.csv : 2829 | lignes users.csv : 50
```

Figure 3 – Tailles des CSV, extrait des fichiers et nombre de lignes.

La couche silver reproduit les règles du lab Spark (renommage, `trim`/`lower`, découpage de l'adresse par expressions régulières, dédoublonnage), mais en Python, puis les tables sont écrites en Iceberg via Trino et Polaris (script en annexe B). Conséquences, à garder en tête :

- pas de Spark : les snapshots de `silver` sont écrits par Trino, et non par Spark ;
- la partie « Compare with Spark » du lab 2 et l'exercice 5 du lab 3 n'ont pas été exécutés ;
- les dates de naissance diffèrent de l'exemple du lab (version de Faker), sans effet sur les résultats.

II. Lab 2 : de silver à gold avec Trino

A. Connexion, catalogues et types

Problème : se connecter à Trino depuis Python et repérer les tables silver. **Solution :** client `trino` (DB-API) et une fonction `sql()` qui exécute puis récupère les résultats dans un `DataFrame`. **Résultat :** Trino 480, six catalogues dont `lakehouse`, deux tables dans `lakehouse.silver` (Figure 4).

python — connexion et catalogues

```
>>> sql("SELECT version()")
  _col0
0   480

>>> sql("SHOW CATALOGS")
     Catalog
0        jmx
1  lakehouse
2     memory
3     system
4      tpcds
5       tpch

>>> sql("SHOW SCHEMAS FROM lakehouse")
              Schema
0  information_schema
1             silver
2             system

>>> sql("SHOW TABLES FROM lakehouse.silver")
    Table
0  orders
1   users
```

Figure 4 – Version, catalogues, schémas et tables.

Le schéma `silver` est bien exposé via Polaris. Les types sont en Figure 5.

python — DESCRIBE des tables silver

```
>>> sql("DESCRIBE lakehouse.silver.orders")
       Column                        Type Extra Comment
0    order_id                     varchar
1     user_id                     varchar
2  ordered_at  timestamp(6) with time zone
3    quantity                     integer
4     product                     varchar

>>> sql("DESCRIBE lakehouse.silver.users")
      Column     Type Extra Comment
0    user_id  varchar
1   username  varchar
2       name  varchar
3        sex  varchar
4      email  varchar
5  birthdate     date
6     street  varchar
7       city  varchar
8      state  varchar
9   zip_code  varchar
```

Figure 5 – Types Trino des tables silver.

Que change `catalog=` ? Il fixe le catalogue par défaut de la session : un nom de table non qualifié est cherché dans `lakehouse`. Ici toutes les requêtes sont qualifiées en `catalogue.schéma.table`, donc le paramètre n'a pas d'effet visible.

Type de `ordered_at` ? `timestamp(6) with time zone` : précision à la microseconde, avec fuseau (les valeurs sortent en `+00:00`).

Type de `order_id` ? `varchar`, comme la colonne string du lab Spark.

B. Requêtes sur la couche silver

Problème : vérifier que silver est cohérente avant de construire gold. **Solution :** comptages, répartition par état (analyse du découpage d'adresse) et recherche des commandes sans utilisateur (`LEFT JOIN` + `IS NULL`). **Résultat :** Figure 6.

python — requêtes sur la couche silver

```
>>> sql("SELECT count(*) AS orders FROM lakehouse.silver.orders")
   orders
0    2829

>>> sql("SELECT count(*) AS users FROM lakehouse.silver.users")
   users
0     50

>>> sql("""
...     SELECT state,
...            count(*) AS users,
...            count_if(city IS NULL) AS missing_city
...     FROM lakehouse.silver.users
...     GROUP BY state
...     ORDER BY users DESC
...     LIMIT 10
... """)
  state  users  missing_city
0    AE      3             3
1    FM      3             0
2    TX      2             0
3    VI      2             0
4    VT      2             0
5    DE      2             0
6    HI      2             0
7    NJ      2             0
8    AP      2             2
9    PA      2             0

>>> sql("""
...     SELECT count(*) AS orphan_orders
...     FROM lakehouse.silver.orders AS o
...     LEFT JOIN lakehouse.silver.users AS u ON u.user_id = o.user_id
...     WHERE u.user_id IS NULL
... """)
   orphan_orders
0              0
```

Figure 6 – Comptages, états et commandes orphelines.

- 2 829 commandes et 50 utilisateurs : 50 est le nombre attendu par le lab Spark.
- Les états `AE` (3 utilisateurs, 3 sans ville) et `AP` (2, 2 sans ville) sont des adresses militaires (`APO`/`DPO`) sans ville : le `NULL` sur `city` est voulu.
- **0 commande orpheline** : chaque `user_id` de `orders` existe dans `users`.

Quel test dbt cette requête remplace-t-elle ? Le test `relationships` (clé étrangère `orders.user_id` vers `users.user_id`). Pour l'exécuter à chaque rafraîchissement, on l'enchaîne après la transformation dans un job qui échoue si `orphan_orders > 0`.

Que dit l'absence de session ou de job ? La requête est exécutée immédiatement par le coordinateur, sans soumission : Trino convient aux requêtes interactives ou ad hoc, alors que Spark a un coût de démarrage de session.

C. Métadonnées Iceberg

Les tables cachées `$snapshots`, `$history` et `$files` donnent l'historique et les fichiers (Figures 7 et 8).

```
python — métadonnées Iceberg de silver.orders

>>> sql("""
...     SELECT snapshot_id, committed_at, operation
...     FROM lakehouse.silver."orders$snapshots"
...     ORDER BY committed_at
... """)
          snapshot_id                committed_at operation
0  6268509190416712600 2026-11-09 10:39:45.436+00:00    append
1  3437261384581602223 2026-11-09 10:39:48.760+00:00    append
2  6703328566673608371 2026-11-09 10:39:49.412+00:00    append
3  1761484694123949566 2026-11-09 10:39:50.075+00:00    append
4  1179259204340169886 2026-11-09 10:39:50.599+00:00    append
5  7043933791627397294 2026-11-09 10:39:51.178+00:00    append
6  7267122905627920030 2026-11-09 10:39:51.624+00:00    append

>>> sql('SELECT * FROM lakehouse.silver."orders$history"')
               made_current_at          snapshot_id     parent_id  is_current_ancestor
0 2026-11-09 10:39:45.436+00:00  6268509190416712600           NaN                 True
1 2026-11-09 10:39:48.760+00:00  3437261384581602223  6.268509e+18                 True
2 2026-11-09 10:39:49.412+00:00  6703328566673608371  3.437261e+18                 True
3 2026-11-09 10:39:50.075+00:00  1761484694123949566  6.703329e+18                 True
4 2026-11-09 10:39:50.599+00:00  1179259204340169886  1.761485e+18                 True
5 2026-11-09 10:39:51.178+00:00  7043933791627397294  1.179259e+18                 True
6 2026-11-09 10:39:51.624+00:00  7267122905627920030  7.043934e+18                 True
```

Figure 7 – Snapshots et historique de `silver.orders`.

Combien de snapshots, écrits par quel moteur ? **7 snapshots**, tous de type `append`, écrits par **Trino** (la création de la table, puis 6 insertions de 500 lignes au plus). Dans le lab d'origine, ce serait Spark (3 snapshots dans l'exemple) : c'est un écart de ma réplique.

```
python + aws cli — fichiers de silver.orders

>>> sql("""
...     SELECT file_path, file_format, record_count, file_size_in_bytes
...     FROM lakehouse.silver."orders$files"
... """)
                                                
```

python — EXPLAIN (TYPE DISTRIBUTED)

```
>>> plan = sql("""
...     EXPLAIN (TYPE DISTRIBUTED)
...     SELECT u.state, count(*) AS orders
...     FROM lakehouse.silver.orders AS o
...     JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
...     GROUP BY u.state
... """)
... print(plan.iloc[0, 0])
Trino version: 480
Fragment 0 [HASH]
    Output layout: [state, count]
    Output partitioning: SINGLE []
    Output[columnNames = [state, orders]]
    │   Layout: [state:varchar, count:bigint]
    │   Estimates: {rows: 37 (944B), cpu: 0, memory: 0B, network: 0B}
    │   orders := count
    └─ Aggregate[type = FINAL, keys = [state]]
       │   Layout: [state:varchar, count:bigint]
       │   Estimates: {rows: 37 (944B), cpu: 70.45k, memory: 944B, network: 0B}
       │   count := count(count_4)
       └─ LocalExchange[partitioning = HASH, arguments = [state::varchar]]
          │   Layout: [state:varchar, count_4:bigint]
          │   Estimates: {rows: 2829 (70.45kB), cpu: 70.45k, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [1]]
                Layout: [state:varchar, count_4:bigint]

Fragment 1 [SOURCE]
    Output layout: [state, count_4]
    Output partitioning: HASH [state]
    Aggregate[type = INTERMEDIATE, keys = [state]]
    │   Layout: [state:varchar, count_4:bigint]
    │   Estimates: {rows: 2829 (70.45kB), cpu: ?, memory: ?, network: ?}
    │   count_4 := count(count_4)
    └─ InnerJoin[criteria = (user_id = user_id_0), distribution = REPLICATED]
       │   Layout: [count_4:bigint, state:varchar]
       │   Estimates: {rows: 2829 (70.45kB), cpu: 117.77k, memory: 3.88kB, network: 0B}
       │   Distribution: REPLICATED
       │   dynamicFilterAssignments = {user_id_0 -> #df_318}
       ├─ Aggregate[type = PARTIAL, keys = [user_id]]
       │  │   Layout: [user_id:varchar, count_4:bigint]
       │  │   Estimates: {rows: 2829 (43.44kB), cpu: ?, memory: ?, network: ?}
       │  │   count_4 := count(*)
       │  └─ ScanFilter[table = lakehouse:silver.orders$data@7267122905627920030, dynamicFilters = {user_id = #df_318}]
       │        Layout: [user_id:varchar]
       │        Estimates: {rows: 2829 (18.58kB), cpu: 18.58k, memory: 0B, network: 0B}/{rows: 2829 (18.58kB), cpu: 18.58k, memory: 0B, network: 0B}
       │        user_id := 2:user_id:varchar
       └─ LocalExchange[partitioning = SINGLE]
          │   Layout: [user_id_0:varchar, state:varchar]
          │   Estimates: {rows: 50 (3.88kB), cpu: 0, memory: 0B, network: 0B}
          └─ RemoteSource[sourceFragmentIds = [2]]
                Layout: [user_id_0:varchar, state:varchar]

Fragment 2 [SOURCE]
    Output layout: [user_id_0, state]
    Output partitioning: BROADCAST []
    TableScan[table = lakehouse:silver.users$data@1555790021435486080]
        Layout: [user_id_0:varchar, state:varchar]
        Estimates: {rows: 50 (3.88kB), cpu: 3.88k, memory: 0B, network: 0B}
        state := 9:state:varchar
        user_id_0 := 1:user_id:varchar
```

Figure 9 – `EXPLAIN (TYPE DISTRIBUTED)` de la jointure.

python — EXPLAIN ANALYZE (extrait)

```
>>> stats = sql("""
...     EXPLAIN ANALYZE
...     SELECT u.state, count(*) AS orders
...     FROM lakehouse.silver.orders AS o
...     JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
...     GROUP BY u.state
... """)
... print(stats.iloc[0, 0])
Trino version: 480
Queued: 1.07ms, Analysis: 98.46ms, Planning: 77.23ms, Execution: 480.65ms, Finishing: 0.00ns
Fragment 1 [HASH]
    CPU: 7.20ms, Scheduled: 12.20ms, Blocked 2.67s (Input: 1.29s, Output: 0.00ns), Input: 52 rows (832B); per task: avg.: 52.00 std.dev.: 0.00, Output: 37 rows (592B)
    Peak Memory: 1.25kB, Tasks count: 1; per task: max: 1.25kB
Fragment 2 [SOURCE]
    CPU: 64.79ms, Scheduled: 167.21ms, Blocked 384.97ms (Input: 138.59ms, Output: 0.00ns), Input: 2879 rows (126.64kB); per task: avg.: 2879.00 std.dev.: 0.00, Output: 52 row…
    Peak Memory: 128kB, Tasks count: 1; per task: max: 128kB
    └─ InnerJoin[criteria = (user_id = user_id_0), distribution = REPLICATED]
       │   Distribution: REPLICATED
       │         Input: 2829 rows (124.30kB), Filtered: 0.00%, Physical input: 83.08kB, Physical input time: 63.51us, Splits: 6, Splits generation wait time: 129.96ms
       │         Dynamic filters:
Fragment 3 [SOURCE]
    CPU: 4.91ms, Scheduled: 19.74ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 50 rows (2.54kB); per task: avg.: 50.00 std.dev.: 0.00, Output: 50 rows (2.54kB)
    Peak Memory: 10.50kB, Tasks count: 1; per task: max: 10.50kB
        Input: 50 rows (2.54kB), Physical input: 5.74kB, Physical input time: 8.51us, Splits: 1, Splits generation wait time: 73.89ms
[…]
```

Figure 10 – `EXPLAIN ANALYZE` (extrait des lignes utiles).

Avec `EXPLAIN ANALYZE` (Figure 10), l'exécution fait apparaître un fragment de plus (1 à 3) : le fragment 2 lit 2 879 lignes (2 829 + 50) avec 64,8 ms de CPU et 128 ko de mémoire de pointe, le fragment 3 lit les 50 utilisateurs. Le scan d'`orders` utilise 6 splits, soit un par fichier Parquet. Le filtre dynamique

ne supprime rien (`Filtered: 0%`), puisque tous les utilisateurs ont des commandes. À cette taille, l'analyse et la planification (98 + 77 ms) pèsent presque autant que l'exécution (481 ms).

```
python — system.runtime.queries

>>> sql("""
...     SELECT query_id, state, date_diff('millisecond', created, coalesce("end", current_timestamp)) AS elapsed_ms, query
...     FROM system.runtime.queries
...     ORDER BY created DESC
...     LIMIT 5
... """)
                    query_id     state  elapsed_ms                                                
```

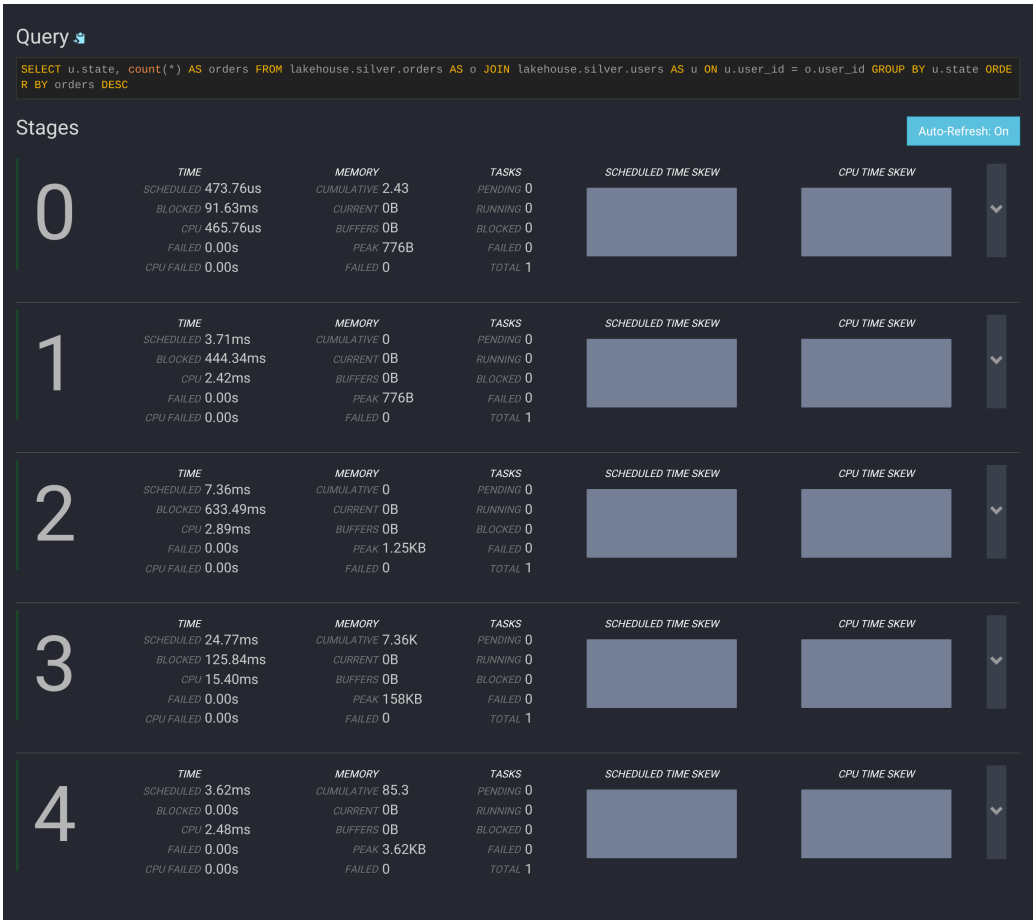


Figure 13 – Interface web Trino : stages de la requête.

F. Validation

Solution : comparer les agrégats de gold à silver avec des `assert`. **Résultat :** aucune `AssertionError` (Figure 18) : 8 615 unités dans silver, dans `orders_by_state_month` et dans `top_products` ; 50 utilisateurs et 2 829 commandes retrouvés dans `user_activity`. La couche gold est cohérente avec silver.

G. Rafraîchissement et time travel

Solution : vider `top_products` par `DELETE`, la remplir par `INSERT`, puis lire la table au premier snapshot (Figure 19).

Combien de snapshots après le rafraîchissement ? 3 : `append` (création), `delete`, `append`. Après le `DELETE`, la table contient 0 ligne ; après l'`INSERT`, 6. Le time travel sur le premier snapshot renvoie bien 6 produits.

Que voit un tableau de bord entre les deux ? Une table **vide** : chaque instruction est un commit séparé. Ici la fenêtre était d'environ une demi-seconde (0,5 s entre les deux commits), mais elle existe. Avec un `MERGE`, la mise à jour est un seul commit : le lecteur voit l'ancien état ou le nouveau, jamais un état vide.

H. Comparaison avec Spark

Cette partie n'a pas été exécutée (pas de Spark dans mon environnement). Réponses de principe, tirées de ce que j'ai observé côté Trino et de ce que demande le lab Spark :

- **Besoins au-delà de la requête.** Trino : un client Python (`trino`) et une connexion, rien

```
python — orders_by_state_month

>>> sql("CREATE SCHEMA IF NOT EXISTS lakehouse.gold")
Empty DataFrame
Columns: []
Index: []

>>> sql("SHOW SCHEMAS FROM lakehouse")
              Schema
0               gold
1  information_schema
2             silver
3             system

>>> sql("""
...     CREATE TABLE lakehouse.gold.orders_by_state_month AS
...     SELECT
...         u.state,
...         date_trunc('month', o.ordered_at) AS month,
...         count(*)                          AS orders,
...         sum(o.quantity)                   AS units,
...         count(DISTINCT o.user_id)         AS buyers
...     FROM lakehouse.silver.orders AS o
...     JOIN lakehouse.silver.users  AS u ON u.user_id = o.user_id
...     GROUP BY 1, 2
... """)
   rows
0    50

>>> sql("SELECT * FROM lakehouse.gold.orders_by_state_month ORDER BY month, state LIMIT 10")
  state                     month  orders  units  buyers
0    AL 2020-01-01 00:00:00+00:00      65    199       1
1    AP 2020-01-01 00:00:00+00:00      17     49       1
2    AS 2020-01-01 00:00:00+00:00      12     40       1
3    CO 2020-01-01 00:00:00+00:00      17     43       1
4    FM 2020-01-01 00:00:00+00:00      66    200       1
5    GA 2020-01-01 00:00:00+00:00       4     12       1
6    IN 2020-01-01 00:00:00+00:00      49    175       1
7    KY 2020-01-01 00:00:00+00:00      83    256       1
8    MH 2020-01-01 00:00:00+00:00      21     57       1
9    NE 2020-01-01 00:00:00+00:00      11     30       1
```

Figure 14 – Schéma `gold` et table `orders_by_state_month` (50 lignes).

```
python — top_products

>>> sql("""
...     CREATE TABLE lakehouse.gold.top_products AS
...     SELECT
...         product,
...         count(*)      AS orders,
...         sum(quantity) AS units,
...         rank() OVER (ORDER BY sum(quantity) DESC) AS units_rank
...     FROM lakehouse.silver.orders
...     GROUP BY product
... """)
   rows
0     6

>>> sql("SELECT * FROM lakehouse.gold.top_products ORDER BY units_rank LIMIT 10")
     product  orders  units  units_rank
0  croissant     491   1523           1
1      drink     484   1493           2
2    brioche     468   1424           3
3      bread     458   1420           4
4      donut     462   1390           5
5     cookie     466   1365           6
```

Figure 15 – Table `top_products` (6 lignes).

d'autre. Spark : une session, le jar Iceberg, `hadoop-aws` et des identifiants S3 temporaires.

- **Tableau de bord toutes les minutes :** Trino (latence faible). **Transformation nocturne de 10 To :** Spark (reprise sur perte de partition). **Exploration de silver par un data scientist :** Trino.
- **Tolérance aux pannes.** Si un worker est perdu, Trino fait échouer la requête. C'est acceptable ici (requêtes de moins d'une seconde, relancées sans coût) mais pas pour une transformation de 10 To qui dure des heures et serait à refaire entièrement.

```
python — user_activity

>>> sql("""
...     CREATE TABLE lakehouse.gold.user_activity AS
...     SELECT
...         u.user_id,
...         u.state,
...         date_diff('year', u.birthdate, current_date) AS age,
...         count(o.order_id)        AS orders,
...         coalesce(sum(o.quantity), 0) AS units,
...         min(o.ordered_at)        AS first_order_at,
...         max(o.ordered_at)        AS last_order_at
...     FROM lakehouse.silver.users AS u
...     LEFT JOIN lakehouse.silver.orders AS o ON o.user_id = u.user_id
...     GROUP BY u.user_id, u.state, u.birthdate
... """)
   rows
0    50

>>> sql("SELECT * FROM lakehouse.gold.user_activity ORDER BY orders DESC LIMIT 10")
                                user_id state  age  orders  units                first_order_at                 last_order_at
0  9793b9b4-1374-4146-b2c5-bd89b70b3420    HI   65     100    289 2020-02-06 22:28:23.377915+00:00 2020-02-11 01:57:32.773432+00:00
1  2051acef-097a-4e10-b6fe-bc0e7ecddbaf    VI    9     100    301 2020-03-13 06:21:16.892555+00:00 2020-03-17 09:02:08.476647+00:00
2  8d605936-0380-4b70-8d03-c91e4f8d5238    WA   76      99    280 2020-03-06 12:42:24.432345+00:00 2020-03-10 14:46:03.127489+00:00
3  52828d80-44b5-41f7-97ac-6aa8bb2488a3    IA   35      97    303 2020-02-11 02:16:02.488268+00:00 2020-02-15 02:18:44.854831+00:00
4  72bb912d-7da6-4785-b63b-4dc3a559e463    NJ   74      97    317 2020-02-15 03:39:36.282921+00:00 2020-02-19 03:23:18.610446+00:00
5  0dde29a6-baa4-471a-9d24-67ac778eedb3    VI   85      92    285 2020-01-08 11:20:45.094193+00:00 2020-01-12 06:23:02.514874+00:00
6  97fa7f04-8363-4007-9cae-9610c72c1fe3    AP   52      92    285 2020-03-27 11:09:51.257559+00:00 2020-03-31 06:01:03.068375+00:00
7  832920b7-c143-4426-b72f-871a45ee432d    HI   51      91    272 2020-04-08 18:29:41.913897+00:00 2020-04-12 12:41:22.197741+00:00
8  90164161-cfa7-41cd-a631-d00b26d794d3    CA   82      91    270 2020-02-22 14:30:47.467747+00:00 2020-02-26 08:13:22.890319+00:00
9  d045dd1c-6684-49e3-b1f8-343ea99f1318    UT   33      86    256 2020-03-02 22:15:57.890701+00:00 2020-03-06 11:03:10.312652+00:00
```

Figure 16 – Table `user_activity` (50 lignes) : les 10 utilisateurs les plus actifs.

```
python — tables et fichiers du schéma gold

>>> sql("SHOW TABLES FROM lakehouse.gold")
                Table
0  orders_by_state_month
1           top_products
2          user_activity

>>> sql("""
...     SELECT file_path, record_count, file_size_in_bytes
...     FROM lakehouse.gold."top_products$files"
... """)
                                                
```

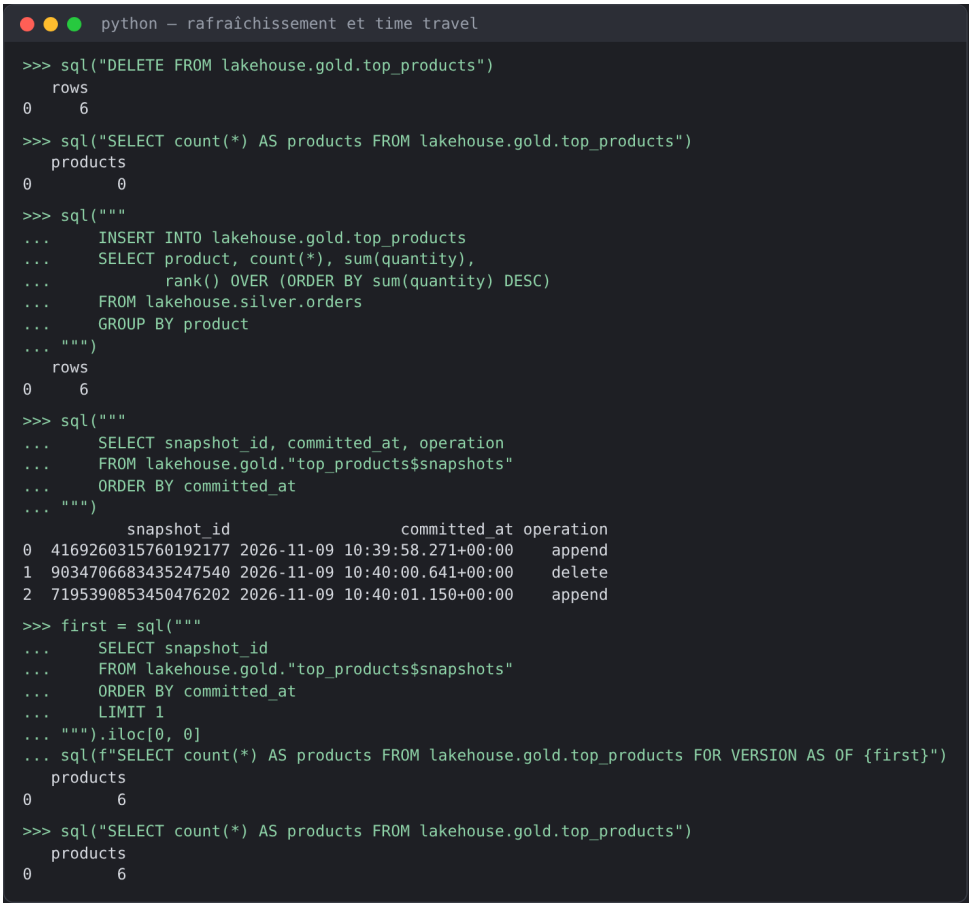


Figure 19 – `DELETE`, `INSERT`, snapshots et `FOR VERSION AS OF`.

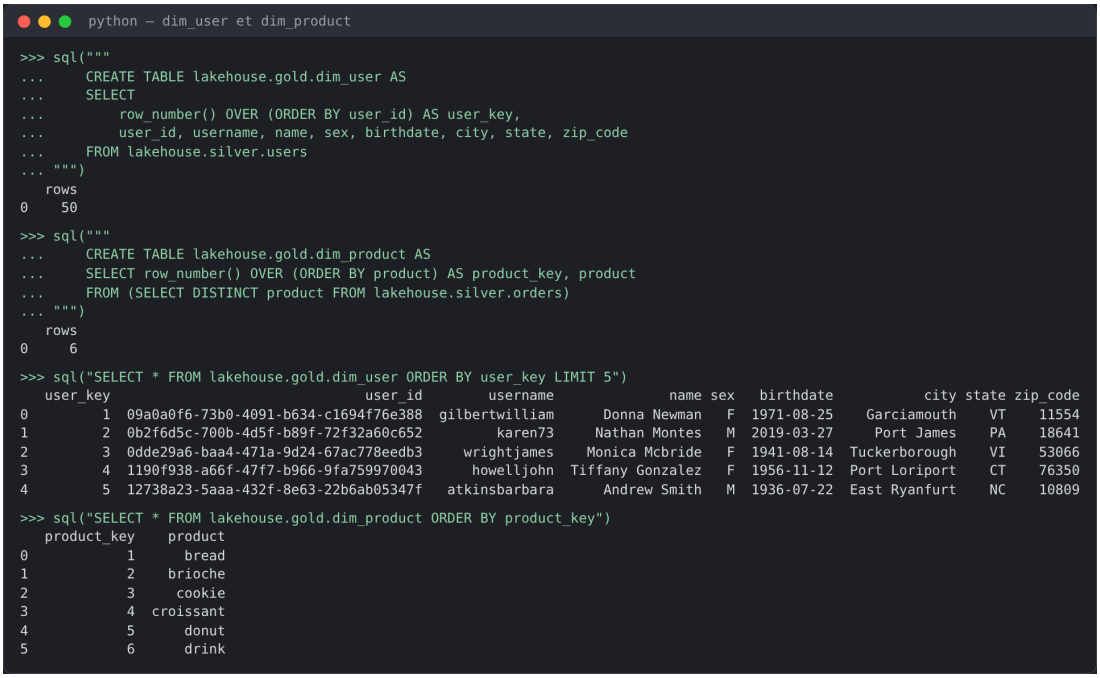


Figure 20 – `dim_user` (50 lignes) et `dim_product` (6 lignes).

B. Table de faits

`fact_orders` garde une ligne par commande, avec les clés des dimensions, obtenues par `LEFT JOIN` (Figure 22) : 2 829 lignes, comme `silver.orders`. Les clés sont des `bigint` (`row_number`) et `date_key`

```
python — dim_date

>>> first_day, last_day = sql("""
...     SELECT CAST(min(ordered_at) AS date) AS first_day,
...            CAST(max(ordered_at) AS date) AS last_day
...     FROM lakehouse.silver.orders
... """).iloc[0]
... print(first_day, last_day)
2020-01-01 2020-04-27

>>> sql(f"""
...     CREATE TABLE lakehouse.gold.dim_date AS
...     SELECT
...         CAST(date_format(CAST(d AS timestamp), '%Y%m%d') AS integer) AS date_key,
...         d                                                AS full_date,
...         year(d)                                          AS year,
...         quarter(d)                                       AS quarter,
...         month(d)                                         AS month,
...         date_format(CAST(d AS timestamp), '%M') AS month_name,
...         day_of_week(d)                                   AS day_of_week,
...         day_of_week(d) IN (6, 7)                         AS is_weekend
...     FROM UNNEST(sequence(DATE '{first_day}', DATE '{last_day}', INTERVAL '1' DAY)) AS t(d)
... """)
   rows
0   118

>>> sql("SELECT * FROM lakehouse.gold.dim_date ORDER BY date_key LIMIT 8")
   date_key   full_date  year  quarter  month month_name  day_of_week  is_weekend
0  20200101  2020-01-01  2020        1      1    January            3       False
1  20200102  2020-01-02  2020        1      1    January            4       False
2  20200103  2020-01-03  2020        1      1    January            5       False
3  20200104  2020-01-04  2020        1      1    January            6        True
4  20200105  2020-01-05  2020        1      1    January            7        True
5  20200106  2020-01-06  2020        1      1    January            1       False
6  20200107  2020-01-07  2020        1      1    January            2       False
7  20200108  2020-01-08  2020        1      1    January            3       False
```

Figure 21 – `dim_date` : 118 jours, du 2020-01-01 au 2020-04-27.

un `integer`.

```
python — fact_orders

>>> sql("""
...     CREATE TABLE lakehouse.gold.fact_orders AS
...     SELECT
...         o.order_id,
...         u.user_key,
...         p.product_key,
...         CAST(date_format(o.ordered_at, '%Y%m%d') AS integer) AS date_key,
...         o.ordered_at,
...         o.quantity
...     FROM lakehouse.silver.orders AS o
...     LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_id = o.user_id
...     LEFT JOIN lakehouse.gold.dim_product AS p ON p.product = o.product
... """)
   rows
0  2829

>>> sql("""
...     SELECT 'dim_user' AS table_name, count(*) AS row_count FROM lakehouse.gold.dim_user
...     UNION ALL SELECT 'dim_product', count(*) FROM lakehouse.gold.dim_product
...     UNION ALL SELECT 'dim_date', count(*) FROM lakehouse.gold.dim_date
...     UNION ALL SELECT 'fact_orders', count(*) FROM lakehouse.gold.fact_orders
... """)
    table_name  row_count
0  fact_orders       2829
1     dim_date        118
2     dim_user         50
3  dim_product          6

>>> sql("DESCRIBE lakehouse.gold.fact_orders")
        Column                      Type Extra Comment
0     order_id                   varchar
1     user_key                    bigint
2  product_key                    bigint
3     date_key                   integer
4   ordered_at  timestamp(6) with time zone
5     quantity                   integer

>>> sql("SELECT * FROM lakehouse.gold.fact_orders ORDER BY ordered_at LIMIT 5")
                              order_id  user_key  product_key  date_key                  ordered_at  quantity
0  2339ba19-2563-4cc3-8a97-ebf555d596af        37            6  20200101 2020-01-01 00:02:17.891488+00:00         4
1  b49e04cc-c243-49e7-a387-eaf6d4c3a832        37            1  20200101 2020-01-01 01:28:08.796458+00:00         5
2  41843b03-04dd-4054-9448-23f7e7e646c7        37            5  20200101 2020-01-01 02:12:56.357080+00:00         2
3  871c0884-9bdf-40f2-ba2c-0f19f0b2a5d1        37            2  20200101 2020-01-01 03:25:23.717132+00:00         3
4  3d0a5f0c-0c39-4ec7-9f31-74054d183eba        37            4  20200101 2020-01-01 04:25:11.371958+00:00         5
```

Figure 22 – Comptage des tables de l'étoile, schéma et extrait de `fact_orders`.

C. Requêtes sur l'étoile

Les requêtes de la Figure 23 joignent le fait aux seules dimensions utiles. La seconde reconstruit `orders_by_state_month` sans lire silver ; la comparaison avec la vraie table de gold renvoie `True` : les 10 premières lignes (commandes, unités, acheteurs) sont identiques.

```
python — requêtes sur l'étoile

>>> sql("""
...     SELECT d.year, u.state, p.product,
...            count(*)        AS orders,
...            sum(f.quantity) AS units
...     FROM lakehouse.gold.fact_orders AS f
...     JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
...     JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
...     JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
...     GROUP BY d.year, u.state, p.product
...     ORDER BY units DESC
...     LIMIT 10
... """)
   year state    product  orders  units
0  2020    FM    brioche      43    134
1  2020    HI      bread      35    110
2  2020    VI      bread      37    110
3  2020    VI  croissant      34    107
4  2020    HI  croissant      36    107
5  2020    VI    brioche      33    104
6  2020    FM  croissant      32    102
7  2020    AE      bread      31    100
8  2020    FM      donut      33     98
9  2020    VT      drink      33     96

>>> star = sql("""
...     SELECT u.state, d.year, d.month,
...            count(*)                 AS orders,
...            sum(f.quantity)          AS units,
...            count(DISTINCT f.user_key) AS buyers
...     FROM lakehouse.gold.fact_orders AS f
...     JOIN lakehouse.gold.dim_date    AS d ON d.date_key = f.date_key
...     JOIN lakehouse.gold.dim_user    AS u ON u.user_key = f.user_key
...     GROUP BY u.state, d.year, d.month
...     ORDER BY d.year, d.month, u.state
...     LIMIT 10
... """)
... star
  state  year  month  orders  units  buyers
0    AL  2020      1      65    199       1
1    AP  2020      1      17     49       1
2    AS  2020      1      12     40       1
3    CO  2020      1      17     43       1
4    FM  2020      1      66    200       1
5    GA  2020      1       4     12       1
6    IN  2020      1      49    175       1
7    KY  2020      1      83    256       1
8    MH  2020      1      21     57       1
9    NE  2020      1      11     30       1

>>> ref = sql("SELECT state, orders, units, buyers FROM lakehouse.gold.orders_by_state_month ORDER BY month, state LIMIT 10")
... print(ref.to_string())
... print("Identique a l'etoile :", (ref[["orders","units","buyers"]].values == star[["orders","units","buyers"]].values).all() and (ref.state.values == star.state.values).al…
  state  orders  units  buyers
0    AL      65    199       1
1    AP      17     49       1
2    AS      12     40       1
3    CO      17     43       1
4    FM      66    200       1
5    GA       4     12       1
6    IN      49    175       1
7    KY      83    256       1
8    MH      21     57       1
9    NE      11     30       1
Identique a l'etoile : True
```

Figure 23 – Requêtes sur l'étoile et comparaison avec `orders_by_state_month`.

Comparaison des deux approches. `orders_by_state_month` répond à une seule question, vite (un scan, aucune jointure). L'étoile répond à toute combinaison d'axes (année × état × produit, par exemple) sans nouvelle table, mais coûte des jointures à chaque requête (3 jointures dans la première).

D. Validation

Les trois contrôles de l'énoncé passent sans erreur (Figure 24) : 2 829 commandes et 8 615 unités dans silver et dans le fait, 0 commande sans ligne de dimension complète. J'y ai ajouté un contrôle hors énoncé : en rejoignant le fait à silver par `order_id`, la clé pointe toujours vers le bon utilisateur et le bon produit (0 erreur sur les deux) et les mesures sont identiques.

Ces contrôles suffisent-ils ? Non. Compter les lignes et sommer les quantités ne détecte pas une commande rattachée au **mauvais** utilisateur ou au mauvais produit : si deux clés sont permutées, les totaux restent justes. D'où le contrôle supplémentaire ci-dessus (comparaison ligne à ligne avec silver). On pourrait ajouter l'unicité de `order_id` et des clés de dimension, et l'absence de `NULL` sur les clés.

```
python — validation de l'étoile
>>> silver_orders = sql("SELECT count(*) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... fact_orders = sql("SELECT count(*) AS v FROM lakehouse.gold.fact_orders").iloc[0, 0]
... silver_units = sql("SELECT sum(quantity) AS v FROM lakehouse.silver.orders").iloc[0, 0]
... fact_units = sql("SELECT sum(quantity) AS v FROM lakehouse.gold.fact_orders").iloc[0, 0]
... assert silver_orders == fact_orders, f"{silver_orders} != {fact_orders}"
... assert silver_units == fact_units, f"{silver_units} != {fact_units}"
... unmatched = sql("""
...     SELECT count(*) AS v
...     FROM lakehouse.gold.fact_orders AS f
...     LEFT JOIN lakehouse.gold.dim_user    AS u ON u.user_key    = f.user_key
...     LEFT JOIN lakehouse.gold.dim_product AS p ON p.product_key = f.product_key
...     LEFT JOIN lakehouse.gold.dim_date    AS d ON d.date_key    = f.date_key
...     WHERE u.user_key IS NULL OR p.product_key IS NULL OR d.date_key IS NULL
... """).iloc[0, 0]
... assert unmatched == 0, f"{unmatched} orders without a complete dimension row"
... print(silver_orders, fact_orders, silver_units, fact_units, unmatched)
2829 2829 8615 8615 0
>>> # Controle supplementaire (hors enonce) : la cle pointe-t-elle vers le BON utilisateur / produit ?
... sql("""
...     SELECT
...       count_if(u.user_id <> o.user_id)  AS wrong_user,
...       count_if(p.product <> o.product)  AS wrong_product,
...       count_if(f.ordered_at <> o.ordered_at OR f.quantity <> o.quantity) AS wrong_measure
...     FROM lakehouse.gold.fact_orders f
...     JOIN lakehouse.silver.orders o ON o.order_id = f.order_id
...     JOIN lakehouse.gold.dim_user u ON u.user_key = f.user_key
...     JOIN lakehouse.gold.dim_product p ON p.product_key = f.product_key
... """)
   wrong_user  wrong_product  wrong_measure
0           0              0              0
```

Figure 24 – Validation de l'étoile (`assert`) et contrôle supplémentaire.

Grain. `fact_orders` : une commande. `dim_date` : un jour calendaire.

Quelle assertion casse selon le défaut ? Testé sans rien écrire (Figure 25) :

- `user_id` **en double dans** `dim_user` : la jointure duplique les commandes de cet utilisateur (2 830 devient 2 930 lignes de fait simulées), donc **la première assertion** (nombre de lignes silver contre fait) échoue.
- **Commande sans produit** (`NULL`) : le nombre de lignes reste correct, mais `product_key` est `NULL` pour 1 ligne (`NULL = NULL` est faux en SQL), donc **l'assertion sur les commandes sans dimension complète** échoue.

Clés `row_number()` après ajout d'un utilisateur. Elles ne sont pas stables : en simulant l'ajout d'un utilisateur dont l'identifiant se classe en premier, la clé change pour **les 50 utilisateurs existants** (Figure 25, deux dernières requêtes). Si `dim_user` est reconstruite sans reconstruire `fact_orders`, les commandes pointent vers le mauvais utilisateur : il faut reconstruire le fait avec la dimension, ou utiliser des clés persistantes (hash de `user_id` ou séquence conservée).

Un utilisateur déménage. `dim_user` écrase les attributs : **toutes** ses commandes passées affichent le nouvel état. Garder l'historique demande une dimension à variation lente de type 2 : une ligne par version, avec nouvelle clé de substitution, dates de début et de fin de validité et un indicateur de ligne courante ; le fait référence la version valide à la date de la commande.

E. Exercices

(a) Exercice 1 : partitionnement

`orders_by_state_month_part` est partitionnée par mois (`month(month)`) : 4 partitions et donc 4 fichiers (un par mois), contre 1 fichier pour la table non partitionnée (Figure 26).

```
python — scénarios de défauts (simulés, sans écriture)

>>> # Defaut 1 : un user_id present deux fois dans dim_user (simule, sans ecriture)
... sql("""
...     WITH dim_user_dup AS (
...         SELECT user_key, user_id FROM lakehouse.gold.dim_user
...         UNION ALL
...         SELECT 999, user_id FROM lakehouse.gold.dim_user WHERE user_id = '9793b9b4-1374-4146-b2c5-bd89b70b3420'
...     )
...     SELECT (SELECT count(*) FROM lakehouse.silver.orders) AS silver_orders,
...            count(*) AS fact_orders_simule
...     FROM lakehouse.silver.orders o LEFT JOIN dim_user_dup u ON u.user_id = o.user_id
... """)
   silver_orders  fact_orders_simule
0           2830               2930

>>> # Defaut 2 : une commande sans produit (NULL), simulee sans ecriture
... sql("""
...     WITH orders_null AS (
...         SELECT order_id, product FROM lakehouse.silver.orders
...         UNION ALL SELECT 'x-null-product', CAST(NULL AS varchar)
...     )
...     SELECT count(*) AS orders, count_if(p.product_key IS NULL) AS product_key_null
...     FROM orders_null o LEFT JOIN lakehouse.gold.dim_product p ON p.product = o.product
... """)
   orders  product_key_null
0    2831                1

>>> # Defaut 3 : ajout d'un utilisateur dans dim_user reconstruite avec row_number() (simule)
... sql("""
...     WITH new_dim AS (
...         SELECT row_number() OVER (ORDER BY user_id) AS user_key, user_id
...         FROM (SELECT user_id FROM lakehouse.silver.users
...               UNION ALL SELECT '00000000-0000-0000-0000-00000000000a')
...     )
...     SELECT o.user_id, o.user_key AS cle_actuelle, n.user_key AS cle_apres_reconstruction
...     FROM lakehouse.gold.dim_user o JOIN new_dim n ON n.user_id = o.user_id
...     WHERE o.user_key <> n.user_key ORDER BY o.user_key LIMIT 5
... """)
                              user_id  cle_actuelle  cle_apres_reconstruction
0  09a0a0f6-73b0-4091-b634-c1694f76e388             1                         2
1  0b2f6d5c-700b-4d5f-b89f-72f32a60c652             2                         3
2  0dde29a6-baa4-471a-9d24-67ac778eedb3             3                         4
3  1190f938-a66f-47f7-b966-9fa759970043             4                         5
4  12738a23-5aaa-432f-8e63-22b6ab05347f             5                         6

>>> sql("""
...     WITH new_dim AS (
...         SELECT row_number() OVER (ORDER BY user_id) AS user_key, user_id
...         FROM (SELECT user_id FROM lakehouse.silver.users
...               UNION ALL SELECT '00000000-0000-0000-0000-00000000000a')
...     )
...     SELECT count_if(o.user_key <> n.user_key) AS users_dont_la_cle_change, count(*) AS users
...     FROM lakehouse.gold.dim_user o JOIN new_dim n ON n.user_id = o.user_id
... """)
   users_dont_la_cle_change  users
0                        50     50
```

Figure 25 – Défauts simulés : doublon dans la dimension, produit `NULL`, instabilité des clés (sans écriture).

Avec un filtre sur février 2020 (Figure 27), la table non partitionnée lit **50 lignes** (80 % filtrées) et la partitionnée **10 lignes** : seul le fichier du mois est lu (élagage de partitions). À 50 lignes il n'y a pas de gain de temps mesurable ; l'intérêt apparaît avec de gros volumes.

(b) Exercice 2 : cohortes par décennie de naissance

`gold.cohorts` compte, par décennie, les utilisateurs, leurs commandes et la moyenne d'unités par commande (Figure 28). 11 décennies, de 1910 à 2020 ; les totaux (50 utilisateurs, 2 829 commandes) correspondent à silver. La moyenne varie peu (2,79 à 3,19), les cohortes sont petites (1 à 8 utilisateurs), donc peu significatives.

```
python — exercice 1 : partitionnement

>>> sql("""
...     CREATE TABLE lakehouse.gold.orders_by_state_month_part
...     WITH (partitioning = ARRAY['month(month)'])
...     AS SELECT * FROM lakehouse.gold.orders_by_state_month
... """)
   rows
0    50

>>> sql("""SELECT partition, record_count, file_count FROM lakehouse.gold."orders_by_state_month_part$partitions" ORDER BY partition LIMIT 5""")
           partition  record_count  file_count
0  (month_month: 600)           17           1
1  (month_month: 601)           10           1
2  (month_month: 602)           13           1
3  (month_month: 603)           10           1

>>> sql("""SELECT regexp_extract(file_path, '[^/]+/[^/]+/[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."orders_by_state_month_part$files" ORDER BY fi…
                                                                                                                                                                  file  record_count  file_size_in_bytes
0  data/month_month=2020-01/20261109_104007_00072_y4x8r-c0cab966-721d-474a-bef2-4904ac61ed1c.parquet           17                1052
1  data/month_month=2020-02/20261109_104007_00072_y4x8r-7caf2c41-38d6-43b1-b64c-8c5556fcc8e3.parquet           10                 964
2  data/month_month=2020-03/20261109_104007_00072_y4x8r-ea9c6986-059b-4126-a322-8e5ade76aa60.parquet           13                1002
3  data/month_month=2020-04/20261109_104007_00072_y4x8r-95dfd0e2-5b93-46e0-beac-8bc3651d1693.parquet           10                 967

>>> sql("""SELECT count(*) AS files, sum(record_count) AS rows FROM lakehouse.gold."orders_by_state_month$files" """)
   files  rows
0      1    50

>>> sql("""SELECT count(*) AS files, sum(record_count) AS rows FROM lakehouse.gold."orders_by_state_month_part$files" """)
   files  rows
0      4    50
```

Figure 26 – Table partitionnée : partitions, fichiers et comparaison avec la table non partitionnée.

```
python — exercice 1 : EXPLAIN ANALYZE (extrait)

>>> print(sql("""EXPLAIN ANALYZE SELECT sum(orders) FROM lakehouse.gold.orders_by_state_month WHERE month = TIMESTAMP '2020-02-01 00:00:00 UTC' """).iloc[0,0])
Fragment 2 [SOURCE]
    CPU: 4.46ms, Scheduled: 7.75ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 50 rows (1.07kB); per task: avg.: 50.00 std.dev.: 0.00, Output: 1 row (9B)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month$data@6113142810104331017, filterPredicate = (month = timestamp(6) with time zone '2020-02-01 00:00:00.00…
          Input: 50 rows (1.07kB), Filtered: 80.00%, Physical input: 1.33kB, Physical input time: 8.25us, Splits: 1, Splits generation wait time: 33.47ms
[…]

>>> print(sql("""EXPLAIN ANALYZE SELECT sum(orders) FROM lakehouse.gold.orders_by_state_month_part WHERE month = TIMESTAMP '2020-02-01 00:00:00 UTC' """).iloc[0,0])
Fragment 2 [SOURCE]
    CPU: 6.05ms, Scheduled: 8.13ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 10 rows (220B); per task: avg.: 10.00 std.dev.: 0.00, Output: 1 row (9B)
    └─ ScanFilterProject[table = lakehouse:gold.orders_by_state_month_part$data@896598320417626178, filterPredicate = (month = timestamp(6) with time zone '2020-02-01 00:00:0…
          Input: 10 rows (220B), Filtered: 0.00%, Physical input: 964B, Physical input time: 5.95us, Splits: 1, Splits generation wait time: 21.52ms
[…]
```

Figure 27 – `EXPLAIN ANALYZE` avec filtre sur un mois : sans partition (haut) et avec (bas).

```
python — exercice 2 : cohortes

>>> sql("""
...     CREATE TABLE lakehouse.gold.cohorts AS
...     SELECT
...         (year(u.birthdate) / 10) * 10 AS decade,
...         count(DISTINCT u.user_id)     AS users,
...         count(o.order_id)             AS orders,
...         round(CAST(sum(o.quantity) AS double) / count(o.order_id), 2) AS avg_units_per_order
...     FROM lakehouse.silver.users u
...     LEFT JOIN lakehouse.silver.orders o ON o.user_id = u.user_id
...     GROUP BY 1
... """)
   rows
0    11

>>> sql("SELECT * FROM lakehouse.gold.cohorts ORDER BY decade")
    decade  users  orders  avg_units_per_order
0     1910      1      47                 3.19
1     1930      5     198                 3.14
2     1940      4     273                 3.06
3     1950      6     340                 3.01
4     1960      8     433                 3.02
5     1970      5     369                 2.99
6     1980      2      81                 2.79
7     1990      5     402                 3.07
8     2000      4     154                 2.99
9     2010      5     244                 3.08
10    2020      5     288                 3.12

>>> sql("SELECT sum(users) AS users, sum(orders) AS orders FROM lakehouse.gold.cohorts")
   users  orders
0     50    2829
```

Figure 28 – Table `gold.cohorts`.

(c) Exercice 3 : INSERT, MERGE et time travel

Une commande est ajoutée à `silver.orders` (2 829 devient 2 830), puis `orders_by_state_month` est rafraîchie par `MERGE` : 51 lignes touchées (50 mises à jour, 1 nouvelle combinaison état-mois). La table

a ensuite 2 snapshots (`append`, `overwrite`). Le time travel sur le premier donne 2 829 commandes, la table actuelle 2 830 (Figure 29).

```
python — exercice 3 : INSERT, MERGE et time travel

>>> sql("""
...     INSERT INTO lakehouse.silver.orders
...     SELECT '00000000-0000-0000-0000-000000000001', user_id,
...            TIMESTAMP '2020-02-10 12:00:00 UTC', 3, 'croissant'
...     FROM lakehouse.silver.users ORDER BY user_id LIMIT 1
... """)
   rows
0     1

>>> sql("SELECT count(*) AS orders FROM lakehouse.silver.orders")
   orders
0    2830

>>> sql("""
...     MERGE INTO lakehouse.gold.orders_by_state_month AS t
...     USING (
...         SELECT u.state, date_trunc('month', o.ordered_at) AS month,
...                count(*) AS orders, sum(o.quantity) AS units, count(DISTINCT o.user_id) AS buyers
...         FROM lakehouse.silver.orders o JOIN lakehouse.silver.users u ON u.user_id = o.user_id
...         GROUP BY 1, 2
...     ) AS s
...     ON t.state = s.state AND t.month = s.month
...     WHEN MATCHED THEN UPDATE SET orders = s.orders, units = s.units, buyers = s.buyers
...     WHEN NOT MATCHED THEN INSERT (state, month, orders, units, buyers) VALUES (s.state, s.month, s.orders, s.units, s.buyers)
... """)
   rows
0    51

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."orders_by_state_month$snapshots" ORDER BY committed_at""")
          snapshot_id                committed_at  operation
0  6113142810104331017 2026-11-09 10:39:57.653+00:00     append
1  6571751683273052160 2026-11-09 10:40:10.355+00:00  overwrite

>>> snap = sql("""SELECT snapshot_id FROM lakehouse.gold."orders_by_state_month$snapshots" ORDER BY committed_at LIMIT 1""").iloc[0,0]
... print(sql(f"SELECT sum(orders) AS orders_before FROM lakehouse.gold.orders_by_state_month FOR VERSION AS OF {snap}").to_string())
... print(sql("SELECT sum(orders) AS orders_after FROM lakehouse.gold.orders_by_state_month").to_string())
   orders_before
0           2829
   orders_after
0          2830
```

Figure 29 – `INSERT` dans silver, `MERGE` dans gold et lecture de l'état précédent.

(d) Exercice 4 : `optimize`

Sur `top_products` (Figure 30), `optimize` ne change rien : la table n'a qu'un fichier, il n'y a rien à compacter, et aucun snapshot n'est créé (3 avant, 3 après).

```
python — exercice 4 : optimize sur top_products

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at""")
          snapshot_id                committed_at operation
0  4169260315760192177 2026-11-09 10:39:58.271+00:00    append
1  9034706683435247540 2026-11-09 10:40:00.641+00:00    delete
2  7195390853450476202 2026-11-09 10:40:01.150+00:00    append

>>> sql("""SELECT regexp_extract(file_path, '[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files" """)
                                                  file  record_count  file_size_in_bytes
0  20261109_104000_00047_y4x8r-feca1f48-6fa8-43bf-87b4-20da241cd9bf.parquet             6                 760

>>> sql("ALTER TABLE lakehouse.gold.top_products EXECUTE optimize")
Empty DataFrame
Columns: [rows]
Index: []

>>> sql("""SELECT snapshot_id, committed_at, operation FROM lakehouse.gold."top_products$snapshots" ORDER BY committed_at""")
          snapshot_id                committed_at operation
0  4169260315760192177 2026-11-09 10:39:58.271+00:00    append
1  9034706683435247540 2026-11-09 10:40:00.641+00:00    delete
2  7195390853450476202 2026-11-09 10:40:01.150+00:00    append

>>> sql("""SELECT regexp_extract(file_path, '[^/]+$') AS file, record_count, file_size_in_bytes FROM lakehouse.gold."top_products$files" """)
                                                  file  record_count  file_size_in_bytes
0  20261109_104000_00047_y4x8r-feca1f48-6fa8-43bf-87b4-20da241cd9bf.parquet             6                 760
```

Figure 30 – `optimize` sur `top_products` : aucun changement.

Pour voir un effet, je l'ai appliqué à `orders_by_state_month`, réécrite par le `MERGE` (Figure 31). Avant : 3 entrées dans `$files` (le résumé indique 2 fichiers de données ; la troisième est donc, par déduction, le fichier de suppressions écrit par le `MERGE`). Après : un nouveau snapshot de type `replace` et **1 seul fichier** de 51 lignes ; le contenu est inchangé (51 lignes, 2 830 commandes).

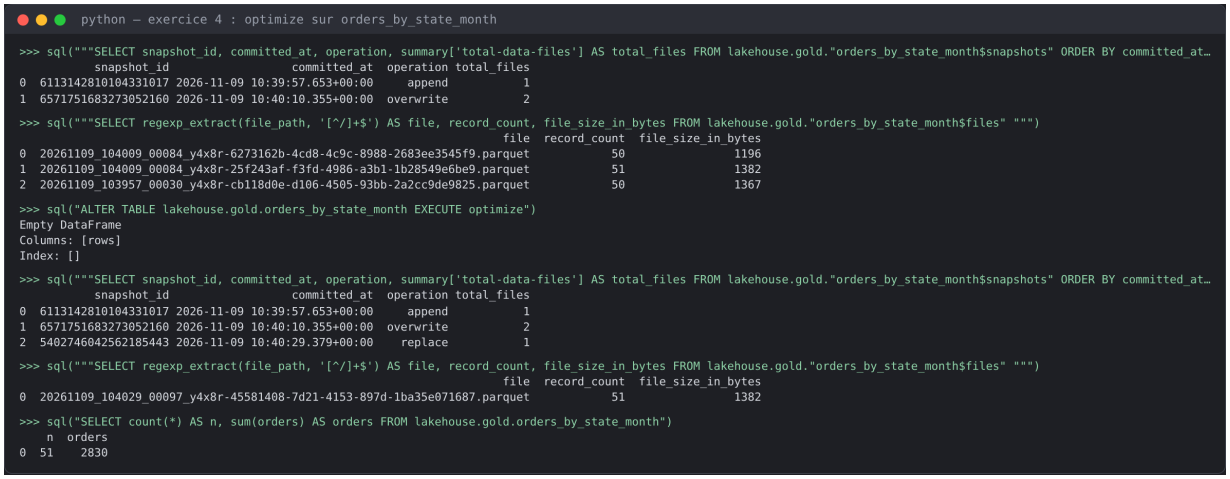


Figure 31 – `optimize` sur `orders_by_state_month` : snapshots et fichiers avant et après.

(e) Exercice 5

Non réalisé : il demande une session Spark connectée à Polaris, absente de mon environnement.

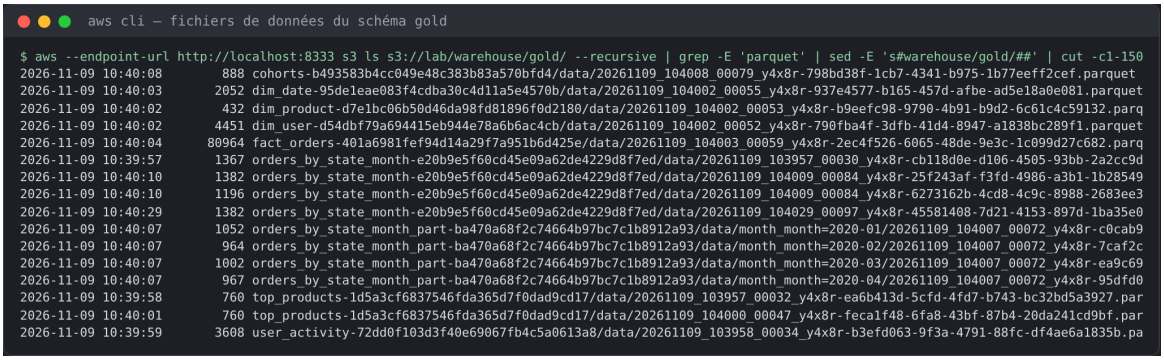


Figure 32 – Fichiers de données du schéma `gold` dans le bucket.

IV. Bilan

- Lab 2 et lab 3 exécutés de bout en bout, sauf la comparaison Spark (lab 2) et l'exercice 5 (lab 3).
- Les résultats sont réels mais obtenus sur une réplique locale (Trino 480, Polaris 1.8.0, SeaweedFS). Les écarts avec l'énoncé viennent de là : silver écrite par Trino (7 snapshots `append`) et non par Spark, et 2 829 commandes au lieu de 5 000.
- Points techniques à retenir : jointure `REPLICATED` choisie par Trino pour une dimension de 50 lignes ; l'élagage de partitions divise par 5 les lignes lues sur l'exemple (50 contre 10) ; `DELETE` puis `INSERT` ouvre une fenêtre à table vide, contrairement à `MERGE` ; les clés `row_number()` changent à la reconstruction.

V. Annexes

A. A. Configuration de la réplique

Listing 1 – docker-compose.yml

```
services:
  s3:
    image: chrislusf/seaweedfs:latest
    command: server -dir=/data -s3 -s3.port=8333 -s3.config=/etc/s3.json -ip=s3
    volumes: ["./s3.json:/etc/s3.json:ro"]
    ports: ["8333:8333"]
  polaris:
    image: apache/polaris:latest
    environment:
      POLARIS_BOOTSTRAP_CREDENTIALS: "POLARIS,root,s3cr3t"
      AWS_REGION: us-east-1
      AWS_ACCESS_KEY_ID: labkey
      AWS_SECRET_ACCESS_KEY: labsecret
      polaris.features."ALLOW_INSECURE_STORAGE_TYPES": "true"
      polaris.features."SUPPORTED_CATALOG_STORAGE_TYPES": '["S3","FILE"]'
      polaris.readiness.ignore-severe-issues: "true"
    ports: ["8181:8181","8182:8182"]
  trino:
    image: trinodb/trino:480
    ports: ["8080:8080"]
    env_file: [trino.env]
    volumes: ["./lakehouse.properties:/etc/trino/catalog/lakehouse.properties:ro"]
    depends_on: [polaris, s3]
```

Listing 2 – Catalogue Trino `lakehouse` (lakehouse.properties)

```
connector.name=iceberg
iceberg.catalog.type=rest
iceberg.rest-catalog.uri=http://polaris:8181/api/catalog
iceberg.rest-catalog.warehouse=lab
iceberg.rest-catalog.security=OAUTH2
iceberg.rest-catalog.oauth2.server-uri=http://polaris:8181/api/catalog/v1/oauth/tokens
iceberg.rest-catalog.oauth2.credential=${ENV:POLARIS_DEV_CREDENTIAL}
iceberg.rest-catalog.oauth2.scope=PRINCIPAL_ROLE:ALL
iceberg.rest-catalog.vended-credentials-enabled=false
fs.native-s3.enabled=true
s3.endpoint=http://s3:8333
s3.region=us-east-1
s3.path-style-access=true
s3.aws-access-key=labkey
s3.aws-secret-key=labsecret
```

B. B. Initialisation du catalogue et de la couche silver

Listing 3 – bootstrap.sh : bucket, catalogue Polaris, rôle, démarrage de Trino

```
set -e
cd /tmp/claude-0/lab; . .venv/bin/activate
docker compose down -v >/dev/null 2>&1
docker compose up -d s3 polaris >/dev/null 2>&1
for i in $(seq 40); do curl -s localhost:8182/q/health | grep -q '"UP"' && break; sleep 2; done
sleep 5
python -c '
import boto3
boto3.client("s3",endpoint_url="http://localhost:8333",aws_access_key_id="labkey",aws_secret_access_key="
    labsecret",region_name="us-east-1").create_bucket(Bucket="lab")'
TOKEN=$(curl -s localhost:8181/api/catalog/v1/oauth/tokens -d grant_type=client_credentials -d client_id=root -d
    client_secret=s3cr3t -d scope=PRINCIPAL_ROLE:ALL | jq -r .access_token)
A="Authorization: Bearer $TOKEN"; J="Content-Type: application/json"
curl -s -o /dev/null -X POST localhost:8181/api/management/v1/catalogs -H "$A" -H "$J" -d '{"catalog":{"name":"
    lab","type":"INTERNAL","readOnly":false,"properties":{"default-base-location":"s3://lab/warehouse"},"
    storageConfigInfo":{"storageType":"S3","allowedLocations":["s3://lab/warehouse"],"endpoint":"http://s3
    :8333","endpointInternal":"http://s3:8333","pathStyleAccess":true,"stsUnavailable":true,"region":"us-east
    -1"}}}'
```

```
curl -s -o /dev/null -X PUT localhost:8181/api/management/v1/principal-roles/service_admin/catalog-roles/lab -H "
    $A" -H "$J" -d '{"catalogRole":{"name":"catalog_admin"}}'
curl -s -o /dev/null -X PUT localhost:8181/api/management/v1/catalogs/lab/catalog-roles/catalog_admin/grants -H "
    $A" -H "$J" -d '{"grant":{"type":"catalog","privilege":"CATALOG_MANAGE_CONTENT"}}'
docker compose up -d trino >/dev/null 2>&1
for i in $(seq 60); do curl -s localhost:8080/v1/info | grep -q '"starting":false' && break; sleep 3; done
sleep 3
python build_silver.py
```

Listing 4 – build_silver.py : transformations silver et écriture Iceberg

```
import re, pandas as pd, trino
u = pd.read_csv("gen/users.csv", dtype=str, keep_default_na=False)
o = pd.read_csv("gen/orders.csv", dtype=str, keep_default_na=False)
lines = u["address"].str.split("\n")
l1 = lines.str[0]; l2 = lines.str[1]
def city(s):
    m = re.match(r"^(.+), [A-Z]{2} [0-9]{5}$", s); return m.group(1) if m else None
def st(s):
    m = re.search(r"([A-Z]{2}) ([0-9]{5})$", s); return (m.group(1), m.group(2)) if m else ("","")
users = pd.DataFrame({
 "user_id":u.uuid,"username":u.username.str.strip(),"name":u.name.str.strip(),
 "sex":u.sex.str.strip().str.upper(),"email":u.mail.str.strip().str.lower(),
 "birthdate":u.birthdate,"street":l1,"city":l2.map(city),
 "state":l2.map(lambda s:st(s)[0]),"zip_code":l2.map(lambda s:st(s)[1])})
orders = pd.DataFrame({"order_id":o.uuid,"user_id":o.user_uuid,"ordered_at":o.date,
 "quantity":o.quantity.astype(int),"product":o["product"].str.strip().str.lower()}).drop_duplicates("order_id")
print(len(users), len(orders))
c = trino.dbapi.connect(host="localhost",port=8080,user="lab",catalog="lakehouse")
def run(q):
    cur=c.cursor(); cur.execute(q); return cur.fetchall()
run("CREATE SCHEMA IF NOT EXISTS lakehouse.silver")
run("""CREATE TABLE IF NOT EXISTS lakehouse.silver.users (user_id varchar, username varchar, name varchar, sex
     varchar, email varchar, birthdate date, street varchar, city varchar, state varchar, zip_code varchar)""")
run("""CREATE TABLE IF NOT EXISTS lakehouse.silver.orders (order_id varchar, user_id varchar, ordered_at
     timestamp(6) with time zone, quantity integer, product varchar)""")
q = lambda v: "NULL" if (v is None or v != v) else "'"+str(v).replace("'","''")+"'"
rows = [f"({q(r.user_id)},{q(r.username)},{q(r['name'])},{q(r.sex)},{q(r.email)},DATE {q(r.birthdate)},{q(r.
     street)},{q(r.city)},{q(r.state)},{q(r.zip_code)})" for _,r in users.iterrows()]
run("INSERT INTO lakehouse.silver.users VALUES "+",".join(rows))
rows = [f"({q(r.order_id)},{q(r.user_id)},TIMESTAMP {q(r.ordered_at.replace(' ','T') if False else r.ordered_at)}
      ,{r.quantity},{q(r['product'])})" for _,r in orders.iterrows()]
for i in range(0,len(rows),500):
    run("INSERT INTO lakehouse.silver.orders VALUES "+",".join(rows[i:i+500]))
print(run("select count(*) from lakehouse.silver.orders"), run("select count(*) from lakehouse.silver.users"))
```